

torch.jit.unused#

<https://pytorch.org/docs/stable/generated/torch.jit.unused.html>

Description:

This decorator indicates to the compiler that a function or method should be ignored and replaced with the raising of an exception. This allows you to leave code in your model that is not yet TorchScript compatible and still export your model. Example (using `@torch.jit.unused` on a method):

Function Signature

```
torch.jit.unused(fn)[source]#
```

Return type

Returns:

Callable[[~_P], ~_R]

Code Examples

```
import torch
import torch.nn as nn

class MyModule(nn.Module):
    def __init__(self, use_memory_efficient):
        super().__init__()
        self.use_memory_efficient = use_memory_efficient

    @torch.jit.unused
    def memory_efficient(self, x):
        import pdb

        pdb.set_trace()
        return x + 10

    def forward(self, x):
        # Use not-yet-scriptable memory efficient mode
        if self.use_memory_efficient:
            return self.memory_efficient(x)
        else:
            return x + 10

m = torch.jit.script(MyModule(use_memory_efficient=False))
m.save("m.pt")

m = torch.jit.script(MyModule(use_memory_efficient=True))
# exception raised
m(torch.rand(100))
```

Detailed Documentation

Documentation

This decorator indicates to the compiler that a function or method should be ignored and replaced with the raising of an exception. This allows you to leave code in your model that is not yet TorchScript compatible and still export your model.

Example (using `@torch.jit.unused` on a method):

This decorator indicates to the compiler that a function or method should be ignored and replaced with the raising of an exception. This allows you to leave code in your model that is not yet TorchScript compatible and still export your model.

Example (using `@torch.jit.unused` on a method):